

APS106 - Fundamentals of Computer Programming

Week 6 | Lecture 1 (6.1) - Midterm Review Jeopardy!

Lecture Structure

1. String Theory
2. To Output OR NOT to Output
3. While, While, While
4. The Good, the Bad, and the Function

1. String Theory

This section contains the solutions for this section focused on strings.

```
In [1]: #Q1 What will this print?
# Simple indexing

fun_string = "APS106 is really fun"
print(fun_string[10:-4]+"!")

# SOLUTION: it prints 'really!'
# python string indexing works by going from the start to end-1 positions of the st
# here, we take all characters from index position 10 (the r in really) up until in
# walking from the back of the string, n is -1, u is -2, f is -3, the blank space i
# so we go from the r to the blank space after really, without including the blank
# therefore, it gets us 'really', which we append an exclamation mark to
```

really!

```
In [2]: #Q2 What will this print?
# more complex indexing w/ step

radio_signal = "Nobody normal ever accomplished anything meaningful in this world."
radio_signal = radio_signal.upper()
radio_signal = radio_signal[15:4:-4].capitalize()
print(radio_signal)

# SOLUTION: it prints 'Van'
# upper() turns all letters in signal to uppercase
# we index [15:4:-4]. So, we take letters from positions 4 to 15: "DY NORMAL EV" an
# we therefore take V A N
# Then, we apply 'capitalize()', which makes the first letter capitalized and all o
# aside: upper() ends up not doing anything meaningful, as capitalize() changes the
```

Van

```
In [3]: #Q3 What will this print?
print("Seb" > "Chris")
print("SEB" > "Seb")
print("Chris" > "Chri")

# SOLUTION:
# True
# False
# True
# Seb > Chris as S falls after C
# SEB < Seb as lowercase are higher values than uppercase in the ASCII table
# Chris > Chri as Chris is a longer string
```

True

False

True

```
In [4]: #Q4 What will this print?
a = "Vecna" > "VecNA"
b = ord('f') - ord('b')
print(a+b)

# SOLUTION: it prints '5'
#a = True/1 as n has a higher ASCII representation than N.
# If a boolean is put into an arithmetic operation, True = 1, False = 0

#b: the difference in order between f and b is 4 (as f is 4 characters above b).
# so ord('f') - ord('b') = 4

# adding these together, the print gives 1 + 4 = 5
```

5

2. To Output OR NOT to Output.

This section contains the solutions for this section focused on boolean operations.

```
In [5]: # Q1 - Multiple choice, select the right answer
# simple boolean operation

# Which of the following evaluates to True when a is equal to b or when a is equal
a = 5
b = 3

#a. a == b == 5
#b. a = b or a = 5
a == b or a == 5
#d. a = b and a = 5
#e. a == b or 5

# SOLUTION: a == b or a == 5
# our two conditions are when a == b ('==' is the comparison operator, '=' is when
# and when a == 5
# a. is bad syntax since you can't have a multiple '==' comparison
```

```
# b. is bad syntax as we want to use '==' for comparisons. This is setting the value
# c. is correct
# d. is the same as b. except with AND
# e. is closer but same issue with a = 5
```

Out[5]: True

```
In [6]: #Q2 - Will it print?
# compound boolean op stuff

a = 10
b = 25
if bool(0) and not a**2 == 100 or b < 50:
    print("Welcome to the real world.")

# SOLUTION: prints "Welcome to the real world."
# bool(0) returns False
# a**2 == 100 i.e., 10*10 == 100 evaluates to True. not a**2 == 100 therefore evaluates to False
# b < 50 i.e., 25 < 50 evaluates to True
# False and False or True evaluates to True, as you have an OR with at least one True
```

Welcome to the real world.

```
In [7]: #Q3 - What will this code print?
# if vs. elif prints

grade1 = 50
grade2 = 100
average = 90

if grade1 >= 90 and grade2 >= 90:
    print("you did exceedingly well on both assignments!")
elif grade2 >= 80:
    print("you did really well on A2!")
if average >= 90:
    print("you did really well in the course!")
elif average >= 70:
    print("you did well in the course")
if average >= 50:
    print("you passed!")

# SOLUTION:
# For the first IF, we don't meet the criteria for the first branch
# but, grade2 >= 90, so we print "you did really well on A2!"
# For the second IF, our average is >= 90, so we print "you did really well in the course!"
# Our average >= 50, so we also print "you passed!".
```

you did really well on A2!
you did really well in the course!
you passed!

```
In [8]: #Q4 - How many times will f() be called?
# lazy evaluation

def f():
    print("f ran")
    return False
```

```

x = 2
if f() and x > 3:
    print("done")
elif x <= 2 or f():
    print ("we're not done yet!")
    f()
    if 2 + x > 4 and f():
        f()

```

SOLUTION: f() is called 2 times
we check the first if statement - f() is called and returns false, so we immediat
we check the elif branch. x <= 2 is true as x was set to 2.
since this condition is an OR and we already have a True, we skip the second f()
since this condition was true, we run f() a second time, then check the next if (
we check 2 + x > 4. as x = 2, 2 + x = 4, 4 is not greater than 4. Since this is a

```

f ran
we're not done yet!
f ran

```

3. While, While, While

This section contains the solutions for this section focused on while loops.

In [9]: *#Q1 - What does this program print?*
Simpler loop calculation with MOD

```

i = 3
j = 12

while j % 3 == 0:
    j = j // 2
    i += 1
    print("i =", i, " j =", j)

```

SOLUTION: below
we Loop until j % 3 != 0, that is when j is not divisible by 3
every loop, we divide j by 2 and increment i by 1
1st loop i ends as 4, j as 6
2nd loop i ends as 5, j as 3
3rd loop i ends as 6, j as 3 // 2 = 1 (1.5 rounded down)
4th loop, j = 1 is not divisible by 3, so we break out of the loop

#Q4 - Loop + compare within loop write (could be final jeopardy)

```

i = 4  j = 6
i = 5  j = 3
i = 6  j = 1

```

In [10]: *#Q2 - How many times does the body of the while-loop get executed?*
infinite loop

```

x = 0
while x < 10:

```

```
x *= 2
x -= 1
```

SOLUTION: infinite times! It never ends
x is multiplied by 2 and decremented by 1 every loop. So x only ever gets smaller

 KeyboardInterrupt

Traceback (most recent call last)

Cell In[10], line 5

```
3 x = 0
4 while x < 10:
----> 5     x *= 2
6     x -= 1
7
8 # SOLUTION: infinite times! It never ends
9 # x is multiplied by 2 and decremented by 1 every loop. So x only ever gets
smaller and will never be larger than +10
```

KeyboardInterrupt:

In [11]: *#Q3 - How many times (i) does loop body run before exiting?*
Loop + compare within loop evaluation

```
string = 'APS10667'
i = 0
while string[i] != string[i+1]:
    i += 1
print(i)
```

SOLUTION: 5
this loop is going through our string 'APS10667' and comparing the i'th character
it'll exit once two consecutive characters are the same
in the 1st loop it compares A and P. A != P, so we go into the loop body
2nd loop it compares P and S. P != S
3rd loop it compares S != 1
4th loop it compares 1 != 0
5th loop it compares 0 != 6
6th loop it compares 6 != 6. This is false as 6 is equal to 6. So we exit after 5

5

In [12]: *# Q4 - What will the contents of the file have? Multiple choice*
Loop + File Access

```
file = open("test_file.txt", "w")
count = 1
while count <= 5:
    file.write(" " * count + str(count) * count + "\n")
    count += 1
file.close()

read_file = open("test_file.txt", 'r')
print(read_file.read())
```

SOLUTION: every time, we add another space in front and add another one of our cu
So, we get 1, then 22, then 333, then 4444, then 55555, slowly shifting over to t

```

1
22
333
4444
55555

```

4. The Good, the Bad, and the Function

This section contains the solutions for this section focused on functions.

```

In [13]: #Q1 - What does n have to be to have this print True?

def calculator(n):
    return n + 3 ** 2 // 2

n = 3
print(calculator(n) == 7.0)

# SOLUTION: n = 3
# working backwards, we need calculator(n) to return 7.0
# we know that calculator(n) returns n + 3 ** 2 // 2 = n + 9 // 2 = n + 4. We need

```

True

```

In [14]: #Q2 - What does this calculate?
# Function calculation basic

def dark_knight_power(n):
    return n ** 3

bat_signal = dark_knight_power(5)
bat_signal += 1

print(bat_signal)

# SOLUTION: 126
# dark_knight_power(n) returns n raised to the power of 3. So n*n*n
# bat_signal is equal to dark_knight_power(5) = 5*5*5 = 125
# then, we increment bat_signal by 1. 125+1=126

```

126

```

In [15]: #Q3 - What is printed?
# print vs return

def printer_1():
    return "Printer 1 online!"
    print("Printer 1 online!")

def printer_2():
    print("Printer 2 online!")

string_1 = printer_1()

```

```
string_2 = printer_2()
print(string_2)
```

```
# SOLUTION: it prints "Printer 2 online", then "None" on the next line
# printer_1 returns before the line "Printer 1 online!" is printed out. So nothing
# printer_2 prints out "Printer 2 online!"
# printer_2 doesn't return anything, so string_2 is assigned a value None, which is
# note that since we use print statements rather than appending these strings toget
```

Printer 2 online!
None

In [16]: #Q4 - What does it print?
functions like crazy

```
def f(n):
    return 3 - n
def g(n):
    return 5 * n
def h(n):
    return 2 * n

def doto(value, func):
    if func == "f":
        return f(value)
    elif func == "g":
        return g(value)
    else:
        return h(value)
```

```
print(doto(doto(2, "g"), "r") + doto(2, "f"))
```

```
# SOLUTION: 21
# going from left to right, the first doto's first argument is another doto. Let's
# doto(2, "g") evaluates to g(2) = 5*2 = 10
# now, we can evaluate doto(doto(2, "g"), "r") as doto(10, "r"). "r" means we'll en
# doto(10, "r") therefore is equal to h(10) = 2*10 = 20
# doto(2, "f") means we call f(2) = 3 - 2 = 1
# we print this doto(10, "r") + doto(2, "f") = 20 + 1 = 21
```

21

5. Final Jeopardy

This section contains the solutions for the Final Jeopardy section.

You want to write a program to process command signals sent by a device in the form "CMDXXXXX" where XXX represent characters in the command. For this program, it matters how many numbers exist in the command.

Write a function that takes in a command string (inputted by the user) and returns a string:

- The function should check if it is a valid command. That is, it should check if the command starts with "CMD".
- The function should count how many numbers appear in the command after CMD.
- The return string should be in the form of "X:Y" where X is either "valid" or "invalid" and Y is the count of numbers found in the command (if invalid, only "invalid" is returned.).

Examples:

- `check_command("CMD:1234Y6")` should return "valid:5"
- `check_command("CMB:1234Y6")` should just return "invalid"

If you have extra time, try adding this feature:

- The program should also append a line to a file "data_logged.txt" that includes the command string and the results of the program, separated by a comma. e.g., for "CMD1234Y6", this should append "CMD1234Y6, valid:5" to the file.

```
In [17]: def check_command(command_string):
    '''
    string -> string
    Takes in command string and returns if it's valid or not.
    If valid, returns string in form of 'valid:X' where X is number of digits in co
    '''
    if command_string.startswith("CMD"):
        i = 0
        count = 0
        while i < len(command_string):
            if command_string[i] in "0123456789":
                count += 1
            i+=1
        result = "valid:" + str(count)
    else:
        result = "invalid"
    return result

command_test = input("Please enter a command string 'CMDXXXXX'")
result = check_command(command_test)
print(result)

# To save the value to file
# opening with 'a' means that we are appending, so we don't overwrite the file ever
file = open("data_logged.txt", "a")
file.write(command_test + "," + result + "\n")
file.close()

# If you want to print your file's data:
file = open("data_logged.txt", "r")
print(file.read())
```

```
valid:3
CMD123RR,valid:3
```



```
In [20]: check_command("CMD:1234Y6")
```

```
Out[20]: 'valid:5'
```

```
In [19]: check_command("CMB:1234Y6")
```

```
Out[19]: 'invalid'
```

```
In [ ]:
```